

# Introduction to Algorithms

Sorting, Recursion, Search, Dynamic Programming

---

Hoa T. Vu

# Introduction to Algorithms

---

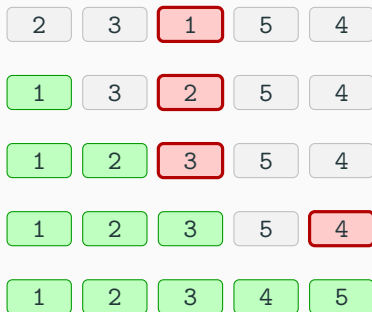
- Sequence of instructions that solve a problem
- Running time: number of computer instructions (memory access and pairwise arithmetics)

# Selection Sort

---

# Selection Sort

- Find the **minimum** element in the unsorted portion
- **Swap** it into the first unsorted position
- Repeat, shrinking the unsorted portion by one each time



## Selection Sort : Code

```
def selection_sort(a):  
    n = len(a)  
    for i in range(n):  
        next_min = i  
        for j in range(i + 1, n):  
            if a[j] < a[next_min]:  
                next_min = j  
        a[i], a[next_min] = a[next_min], a[i]  
  
a = [5, 3, 8, 1, 4]  
selection_sort(a)  
# [1, 3, 4, 5, 8]
```

## Selection Sort : Code

```
def selection_sort(a):  
    n = len(a)                                #1  
    for i in range(n):                        #2  
        next_min = i                         #3  
        for j in range(i + 1, n):           #4  
            if a[j] < a[next_min]:           #5  
                next_min = j                 #6  
        a[i], a[next_min] = a[next_min], a[i] #7
```

## Selection Sort : Running Time

We count the number of times each line runs:

- Lines #1, #3, #7: run  $n$  times (once per outer iteration)
- Lines #4–#6: inner loop runs

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}$$

times in total

- All other lines: constant or  $O(n)$  : negligible

Total  $\approx \frac{n^2}{2}$  operations  $\Rightarrow$  **running time** =  $O(n^2)$ .

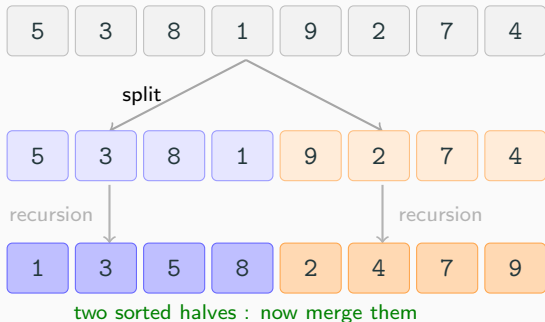


# Merge Sort

---

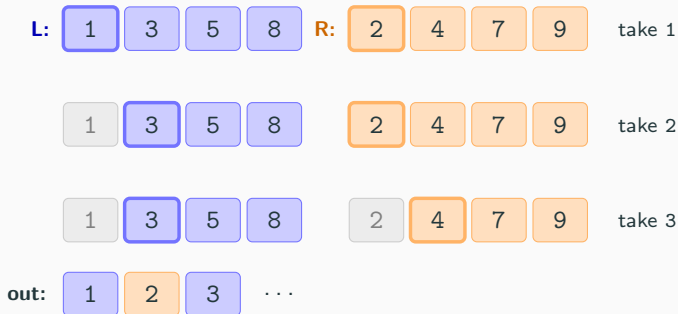
# Merge Sort : Recursion

- **Split** the array into two halves
- **Recursively** sort each half (trust the recursion!)
- **Merge** the two sorted halves (next slide)



## Merge Sort : Merge Step

Compare front elements of L and R; take the smaller one. Repeat.



## Merge Sort : Code (merge)

```
def merge(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out += left[i:]    # append remaining elements
    out += right[j:]
    return out
```

## Merge Sort : Code (merge sort)

```
def merge_sort(a):  
    if len(a) <= 1:          # base case  
        return a  
    mid = len(a) // 2  
    left  = merge_sort(a[:mid])  
    right = merge_sort(a[mid:])  
    return merge(left, right)
```

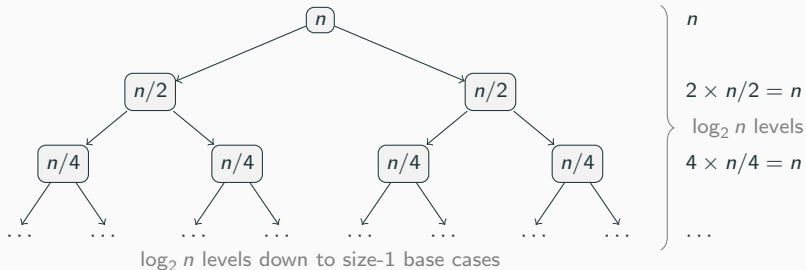
## Merge Sort : Code (merge sort)

```
def merge_sort(a):  
    if len(a) <= 1:           # base case  
        return a  
    mid = len(a) // 2  
    left  = merge_sort(a[:mid])  
    right = merge_sort(a[mid:])  
    return merge(left, right)
```

Merging takes  $n$  time. Hence, we have the recurrence  $T(n) = 2T(n/2) + O(n)$ .

# Merge Sort : Running Time

Each level of the recursion tree does  $O(n)$  total work; there are  $\log_2 n$  levels.



Total work =  $n \times \log_2 n \Rightarrow$  **running time** =  $O(n \log n)$ .

# Tower of Hanoi

---

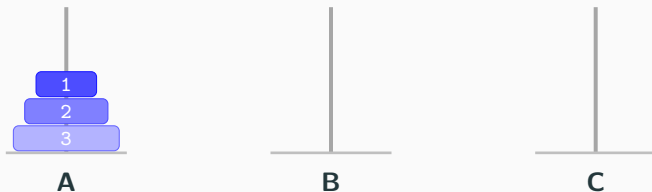


# Tower of Hanoi

**Puzzle:** move  $n$  disks from peg A to peg C using peg B as a helper.

**Rules:**

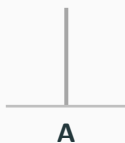
- Move one disk at a time
- Never place a larger disk on top of a smaller one



## Tower of Hanoi : Recursive Idea

**Base case:**  $n = 0$  : no disks to move, do nothing.

**Recursive case:**  $n \geq 1$  : apply the 3-step strategy.



$n = 0$ : nothing to do

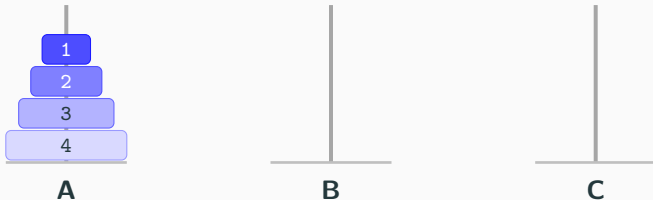


## Tower of Hanoi : Recursive Idea

To move  $n$  disks from  $A \rightarrow C$  using  $B$  as helper:

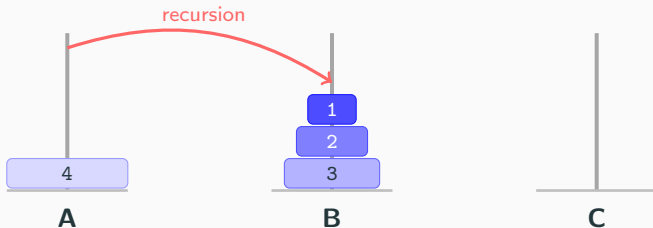
1. Move disks 1 to  $n-1$  from  $A \rightarrow B$  (recursively)
2. Move disk  $n$  (the largest) from  $A \rightarrow C$
3. Move disks 1 to  $n-1$  from  $B \rightarrow C$  (recursively)

*Example:  $n = 4$ , all disks start on A*



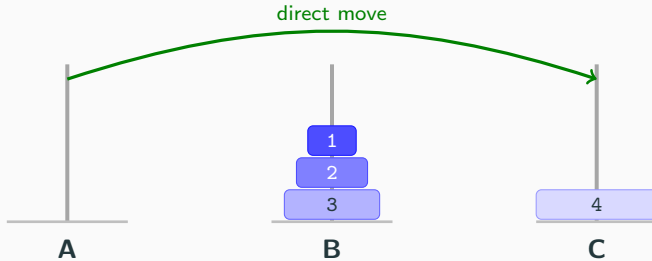
# Tower of Hanoi : Recursive Idea

**Step 1:** move disks 1–3 from A  $\rightarrow$  B (recursively)



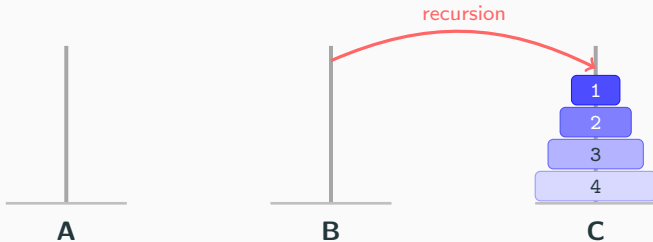
# Tower of Hanoi : Recursive Idea

**Step 2:** move disk 4 from A  $\rightarrow$  C (one direct move)



# Tower of Hanoi : Recursive Idea

**Step 3:** move disks 1–3 from B  $\rightarrow$  C (recursively)  $\Rightarrow$  done!



## Tower of Hanoi : Code

```
def hanoi(n, source, target, helper):  
    if n == 0:  
        return  
    hanoi(n - 1, source, helper, target)    # step 1  
    print(f"Move disk {n}: {source} -> {target}")    # step 2  
    hanoi(n - 1, helper, target, source)    # step 3  
  
hanoi(3, "A", "C", "B")  
# Move disk 1: A -> C           Move disk 1: B -> A  
# Move disk 2: A -> B           Move disk 2: B -> C  
# Move disk 1: C -> B           Move disk 1: A -> C  
# Move disk 3: A -> C
```

## Tower of Hanoi : Running Time

Each call to `hanoi(n, ...)` makes two recursive calls to `hanoi(n-1, ...)` plus one direct move. Let  $T(n)$  be the total number of moves:

$$T(0) = 0, \quad T(n) = 2 T(n - 1) + 1$$



## Tower of Hanoi : Running Time

Each call to `hanoi(n, ...)` makes two recursive calls to `hanoi(n-1, ...)` plus one direct move. Let  $T(n)$  be the total number of moves:

$$T(0) = 0, \quad T(n) = 2 T(n-1) + 1$$

**Unrolling the recurrence:**

$$\begin{aligned} T(n) &= 2 T(n-1) + 1 \\ &= 2(2 T(n-2) + 1) + 1 = 4 T(n-2) + 3 \\ &= 4(2 T(n-3) + 1) + 3 = 8 T(n-3) + 7 \\ &\vdots \\ &= 2^n T(0) + (2^n - 1) = 2^n - 1 \end{aligned}$$

# Binary Search

---

# Binary Search

**Prerequisite:** the array must be **sorted**.

- Compare the target to the **middle** element
- If equal : found
- If target is smaller : search the **left** half
- If target is larger : search the **right** half
- Repeat, halving the search space each time

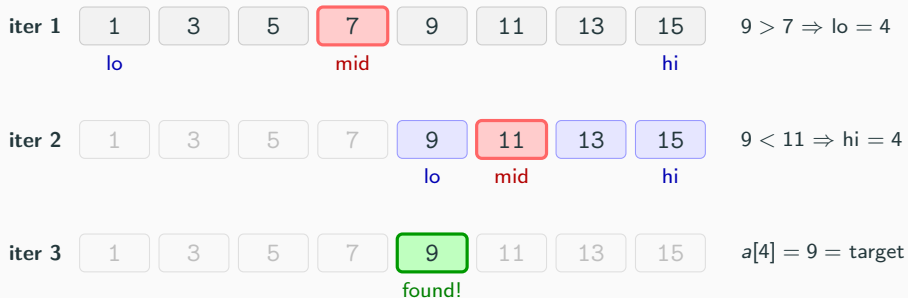


## Binary Search : Code

```
def binary_search(a, target):  
    lo, hi = 0, len(a) - 1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        if a[mid] == target:  
            return mid                # found: return index  
        elif a[mid] < target:  
            lo = mid + 1              # search right half  
        else:  
            hi = mid - 1              # search left half  
    return -1                        # not found
```

- Returns the **index** of target, or  $-1$  if absent
- Each iteration eliminates **half** the remaining elements

## Binary Search : Trace (target = 9)



## Binary Search : Running Time

Each step halves the search space:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow 1$$

After  $k$  steps the search space has size  $n/2^k$ . We stop when  $n/2^k = 1$ , i.e.  $k = \log_2 n$ .

**Running time** =  $O(\log n)$ .

| $n$       | steps |
|-----------|-------|
| 1 000     | 10    |
| 1 000 000 | 20    |
| $10^9$    | 30    |

# Fibonacci

---

# Dynamic Programming : Fibonacci

The Fibonacci sequence is defined by:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2$$

First values:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

**Goal:** given  $n$ , compute  $F(n)$  efficiently.



# Dynamic Programming : Fibonacci

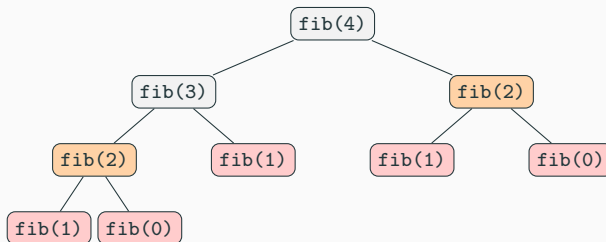
Direct translation of the definition into code:

```
def fib(n):  
    if n <= 1:  
        return n                # base cases:  $F(0)=0$ ,  $F(1)=1$   
    return fib(n-1) + fib(n-2)
```

This is correct, but `fib(n-1)` and `fib(n-2)` each recurse into overlapping subproblems: the same values get recomputed many times.

# Dynamic Programming : Fibonacci

Call tree for `fib(4)`: colored nodes are recomputed:



- `fib(2)` computed  $2\times$ ; `fib(1)` computed  $3\times$ ; `fib(0)` computed  $2\times$
- In general:  $T(n) = T(n-1) + T(n-2) + O(1) \Rightarrow T(n) = O(2^n)$

# Dynamic Programming : Fibonacci (DP)

## Bottom-up DP: $O(n)$

```
def fib_dp(n):  
    dp = [0] * (n + 1)  
    dp[1] = 1  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
    return dp[n]
```

- Fill the table from small to large
- Each entry computed exactly once
- No recursion, no recomputation

## 0/1 Knapsack

---

## 0/1 Knapsack : Problem

Given  $n$  items, each with a weight  $w_i$  and value  $v_i$ , and a knapsack of capacity  $W$ :

**Maximize** total value **s.t.** total weight  $\leq W$ ; each item used **at most once**.

**Example** ( $W = 6$ ):

| Item | Weight | Value |
|------|--------|-------|
| 1    | 1      | 2     |
| 2    | 3      | 4     |
| 3    | 2      | 3     |
| 4    | 4      | 5     |

Which subset of items maximizes total value without exceeding  $W = 6$ ?

## 0/1 Knapsack : Problem

Given  $n$  items, each with a weight  $w_i$  and value  $v_i$ , and a knapsack of capacity  $W$ :

**Maximize** total value **s.t.** total weight  $\leq W$ ; each item used **at most once**.

**Example** ( $W = 6$ ):

| Item | Weight | Value |
|------|--------|-------|
| 1    | 1      | 2     |
| 2    | 3      | 4     |
| 3    | 2      | 3     |
| 4    | 4      | 5     |

Optimal: items 1, 2, 3 (weight  $1 + 3 + 2 = 6$ , value  $2 + 4 + 3 = \mathbf{9}$ ).

## 0/1 Knapsack : Brute Force

Try all  $2^4 = 16$  subsets; keep those with total weight  $\leq 6$ :

Checking all subsets is  $O(2^n)$ : impractical for large  $n$ .

## 0/1 Knapsack : DP Formulation (1/3)

**Subproblem:**  $dp[i][w]$  = max value using the first  $i$  items with weight limit  $w$ .

**Case 1: skip item  $i$ :**

$$value = dp[i-1][w]$$

We simply inherit the best value from the previous  $i - 1$  items, leaving the weight budget unchanged.



## 0/1 Knapsack : DP Formulation (2/3)

**Case 2: take item  $i$**  (only if  $w \geq w_i$ ):

$$value = v_i + dp[i-1][w - w_i]$$

We gain value  $v_i$  and spend weight  $w_i$ , then look up the best value achievable with the remaining capacity  $w - w_i$ .

## 0/1 Knapsack : DP Formulation (3/3)

**Combined recurrence:** take the better of the two cases.

$$\text{dp}[i][w] = \max \left( \underbrace{\text{dp}[i-1][w]}_{\text{skip}}, \underbrace{v_i + \text{dp}[i-1][w-w_i]}_{\text{take, } w \geq w_i} \right)$$

**Base case:**  $\text{dp}[0][w] = 0$  for all  $w$  (no items available  $\Rightarrow$  value is 0).

## 0/1 Knapsack : Code

```
def knapsack(weights, values, W):  
    n = len(weights)  
    dp = [[0] * (W + 1) for _ in range(n + 1)]  
    for i in range(1, n + 1):  
        for w in range(W + 1):  
            dp[i][w] = dp[i-1][w]           # skip item i  
            if w >= weights[i-1]:  
                take = values[i-1] + dp[i-1][w - weights[i-1]]  
                dp[i][w] = max(dp[i][w], take)  
    return dp[n][W]
```

Time:  $O(nW)$     Space:  $O(nW)$

# Longest Increasing Subsequence

---

# Longest Increasing Subsequence

Given a sequence of numbers, find the length of the longest **strictly increasing** subsequence.

A subsequence does **not** need to be contiguous.

## Longest Increasing Subsequence : Example

$a = [10, 9, 2, 5, 3, 7, 101, 18]$

- $9 \rightarrow 101$  (length 2)
- $2 \rightarrow 5 \rightarrow 7 \rightarrow 101$  (length 4)
- $2 \rightarrow 3 \rightarrow 7 \rightarrow 101$  (length 4)  $\leftarrow$  one LIS

LIS length = 4.

## Longest Increasing Subsequence : Subproblem

Instead of asking “what is the LIS of the whole array?”, ask:

$dp[i]$  = length of the LIS that *ends* at index  $i$

**Base case:** every single element is a valid subsequence:

$$dp[i] = 1 \quad \text{for all } i$$

**Answer:**  $\max_i dp[i]$

## Longest Increasing Subsequence : Recurrence

To extend an LIS ending at  $j$  through index  $i$ , we need  $a[j] < a[i]$ :

$$\text{dp}[i] = 1 + \max \{ \text{dp}[j] \mid j < i, a[j] < a[i] \}$$

Scan all previous indices  $j < i$ , keep the best valid one.



## Longest Increasing Subsequence : Example Step

$a = [10, 9, 2, 5, 3, 7, \dots]$     Computing  $dp[3]$  ( $a[3]=5$ ):

- $j = 0$ :  $a[0] = 10 > 5$     skip
- $j = 1$ :  $a[1] = 9 > 5$     skip
- $j = 2$ :  $a[2] = 2 < 5$      $dp[2] + 1 = 2$

$dp[3] = 2$     (the LIS  $2 \rightarrow 5$ )

## Longest Increasing Subsequence : Trace

$a = [10, 9, 2, 5, 3, 7, 101, 18]$

|           |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 10        | 9         | 2         | 5         | 3         | 7         | 101       | 18        |
| $dp[0]=1$ | $dp[1]=1$ | $dp[2]=1$ | $dp[3]=2$ | $dp[4]=2$ | $dp[5]=3$ | $dp[6]=4$ | $dp[7]=4$ |

$LIS\ length = \max(dp) = 4$

## Longest Increasing Subsequence : Code

```
def lis(a):  
    n = len(a)  
    dp = [1] * n  
    for i in range(1, n):  
        for j in range(i):  
            if a[j] < a[i]:  
                dp[i] = max(dp[i], dp[j] + 1)  
    return max(dp)  
  
print(lis([10, 9, 2, 5, 3, 7, 101, 18])) # 4
```

## Longest Increasing Subsequence : Running Time

- Outer loop:  $n$  iterations
- Inner loop: up to  $n$  iterations each
- Total:  $O(n^2)$

Space:  $O(n)$  for the dp array.

An  $O(n \log n)$  algorithm exists using binary search, but  $O(n^2)$  is the natural DP solution.